LAPORAN TUGAS BESAR

Milestone 1: Sistem Terdistribusi Dasar

IF4031 Arsitektur Aplikasi Terdistribusi

Semester I Tahun Akademik 2026/2027

Disusun oleh

Rafif Farras 13523095

Azfa Radhiyya Hakim 13523115

Samantha Laqueenna Ginting 13523138

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Pernyataan Integritas Akademik dan Penggunaan AI

Rafif Farras (13523095)	Azfa Radhiyya Hakim (13523115)	Samantha Laqueenna Ginting (13523138)

Pembagian Tugas

Anggota	Kontribusi
Rafif Farras (13523095)	<Kontribusi Anggota 1>
Azfa Radhiyya Hakim (13523115)	<Kontribusi Anggota 2>
Samantha Laqueenna Ginting (13523138)	<Kontribusi Anggota 3>

Daftar Isi

Pernyataan Integritas Akademik dan Penggunaan AI

Pembagian Tugas

1 Pendahuluan **1**

1.1 Latar Belakang . . . 1
1.2 Tujuan . . . 1
1.3 Ruang Lingkup Milestone 1 . . . 1
1.4 Batasan Sistem . . . 2
1.5 Struktur Laporan . . . 2

2 Arsitektur Sistem **2**

2.1 Gambaran Umum . . . 2
2.2 Komponen dan Batas Service . . . 4
2.3 Alur Komunikasi dan Data . . . 4
2.4 Teknologi dan Konfigurasi . . . 4

3 Problem 1: Arsitektur dan Interoperabilitas Data **5**

3.1 Rancangan Solusi . . . 5
3.1.1 Aturan Pemetaan ke HazardEvent . . . 5
3.1.2 Kebijakan Evolusi Skema . . . 6
3.2 Alternatif dan Trade-off . . . 6
3.3 Proof of Concept (PoC) . . . 7

4 Problem 2: Concurrency, Availability, dan Graceful Degradation **9**

4.1 Rancangan Solusi . . . 9
4.1.1 Tiga Risiko yang Ditangani . . . 10
4.1.2 Model Konkurensi dan Isolasi Sumber . . . 10
4.1.3 Graceful Degradation dan Data Stale . . . 10
4.2 Alternatif dan Trade-off . . . 12
4.3 Proof of Concept (PoC) . . . 12
4.3.1 Uji Latensi BMKG-only . . . 12
4.3.2 Uji Beban Berkelanjutan . . . 13

4.3.3 Outage dan Pemulihan PVMBG 14

5 Problem 3: Service-to-Service Authentication dan Trust 15

5.1 Rancangan Solusi 15

5.2 Alternatif dan Trade-off 15

5.3 Batasan dan Bukti Penyelesaian 15

6 Problem 4: Arsitektur Microservice dan Flexible Storage 15

6.1 Rancangan Solusi 15

6.2 Alternatif dan Trade-off 15

6.3 Batasan dan Bukti Penyelesaian 15

7 Problem 5: Fan-out Event Distribution dan Pub-sub 16

7.1 Rancangan Solusi 16

7.2 Alternatif dan Trade-off 16

7.3 Batasan dan Bukti Penyelesaian 16

A Bukti Pengujian Tambahan 16

1 Pendahuluan

1.1 Latar Belakang

Penanggulangan bencana melibatkan lembaga-lembaga yang bekerja secara otonom, memiliki sumber data, format informasi, dan karakteristik infrastruktur yang berbeda. Dalam skenario tugas ini, BMKG menyediakan informasi kejadian seismik dan potensi tsunami, sedangkan PVMBG menyediakan laporan aktivitas vulkanik. BNPB membutuhkan kedua aliran informasi tersebut untuk membangun gambaran situasi yang terpadu dan menyajikannya kepada pihak yang berwenang. Namun, perbedaan format data, latensi layanan, dan batas kepercayaan antarlembaga membuat integrasi langsung menjadi tidak memadai.

Masalah tersebut tidak hanya berkaitan dengan pertukaran data. Ketika satu layanan sumber melambat atau gagal, layanan lain tetap dapat menghasilkan data yang bernilai bagi pengguna. Demikian pula, lonjakan permintaan pada saat status bencana meningkat tidak boleh membuat seluruh sistem berhenti merespons. Sistem juga perlu membedakan hak akses antara Media, Tim Lapangan, dan BNPB Internal agar data mentah seperti koordinat presisi serta atribut spesifik sumber tidak bocor kepada pihak yang hanya membutuhkan ringkasan.

Oleh sebab itu, proyek ini membangun proof-of-concept platform koordinasi kebencanaan BNPB sebagai aplikasi terdistribusi. Platform menyatukan data heterogen dari BMKG dan PVMBG melalui komunikasi jaringan, mengubahnya ke format kanonik *HazardEvent*, menyimpannya pada *Canonical Store*, dan mendistribusikan informasi tersebut melalui API serta mekanisme *publish-subscribe*.

1.2 Tujuan

Tujuan utama pengembangan sistem adalah membuktikan bahwa beberapa layanan otonom dapat diintegrasikan menjadi satu platform koordinasi tanpa menghilangkan batas kepemilikan dan karakteristik masing-masing layanan. Secara khusus, Milestone 1 bertujuan untuk:

1. menyatukan data seismik dan vulkanik dari dua instansi ke dalam format kanonik yang konsisten serta tetap menerima perubahan skema aditif;
2. menjaga ketersediaan data penting melalui pemrosesan asinkron, pembatasan konkurensi, dan degradasi layanan yang eksplisit ketika salah satu sumber tidak tersedia;
3. menerapkan pemisahan domain kepercayaan untuk komunikasi antarlayanan dan akses client downstream;
4. memisahkan komponen ke dalam layanan dan container yang dapat dijalankan secara independen, dengan penyimpanan yang mendukung atribut dinamis; serta
5. mendistribusikan event baru ke beberapa consumer melalui message broker tanpa membuat Aggregator bergantung langsung pada consumer tersebut.

1.3 Ruang Lingkup Milestone 1

Ruang lingkup implementasi dibatasi pada dua instansi sumber dan dua jenis bencana. BMKG disimulasikan sebagai sumber *SeismicEvent* dan *TsunamiWarning*; PVMBG disimulasikan

sebagai sumber *VolcanicReport*. Kedua sumber bersifat pasif dan hanya merespons polling berkala dari Aggregator BNPB. Data dari sumber tidak dibagikan melalui pemanggilan fungsi langsung atau library *business logic* bersama, melainkan melalui API jaringan sesuai kontrak layanan.

Komponen BNPB dalam Milestone 1 meliputi Aggregator atau gateway, Canonical Store, Client-Facing API, Auth/Token Service, message broker, dan sedikitnya dua event consumer. Aggregator melakukan polling, pemetaan, penyimpanan, serta publikasi event baru. Canonical Store hanya dimiliki dan diakses langsung oleh Aggregator; layanan lain memperoleh data melalui API Aggregator. Client-Facing API bersifat *read-only* dan menerapkan hak akses berdasarkan scope token.

Hak akses client dibagi menjadi tiga tingkat. Media hanya menerima field berklasifikasi Ringkasan. Tim Lapangan memperoleh data lengkap untuk kebutuhan operasional, sedangkan BNPB Internal memiliki hak akses tertinggi. Access token bersifat berumur pendek dan harus dapat diperbarui melalui alur *refresh* tanpa login ulang. Batasan ini dibuat agar keamanan, ketahanan, dan distribusi data dapat diuji secara nyata pada arsitektur yang sama.

1.4 Batasan Sistem

Milestone 1 belum mencakup seluruh kebutuhan sistem kebencanaan nasional. Sistem tidak menangani jalur pelaporan dari lapangan ke BNPB, perubahan data oleh client downstream, maupun sumber bencana lain seperti kebakaran hutan dan hidrometeorologi. Fokusnya adalah fondasi interoperabilitas, availability, keamanan, fleksibilitas storage, dan distribusi event pada skala dasar.

Pengembangan untuk Milestone 2 berada di luar cakupan laporan ini. Tahap berikutnya akan menguji fondasi yang sama dalam kondisi yang lebih skalabel, termasuk banyak instance layanan, volume data yang meningkat drastis, dan kegagalan beberapa komponen secara bersamaan.

1.5 Struktur Laporan

Bab berikutnya menjelaskan arsitektur sistem, batas tiap service, serta teknologi yang digunakan. Setelah itu, setiap problem resmi Milestone 1 dibahas secara terpisah, mencakup rancangan solusi, alternatif dan trade-off, batasan yang disadari, serta bukti penyelesaian. Lampiran digunakan untuk menyimpan bukti pengujian tambahan seperti tangkapan layar, log ber-correlation ID, dan hasil pengukuran.

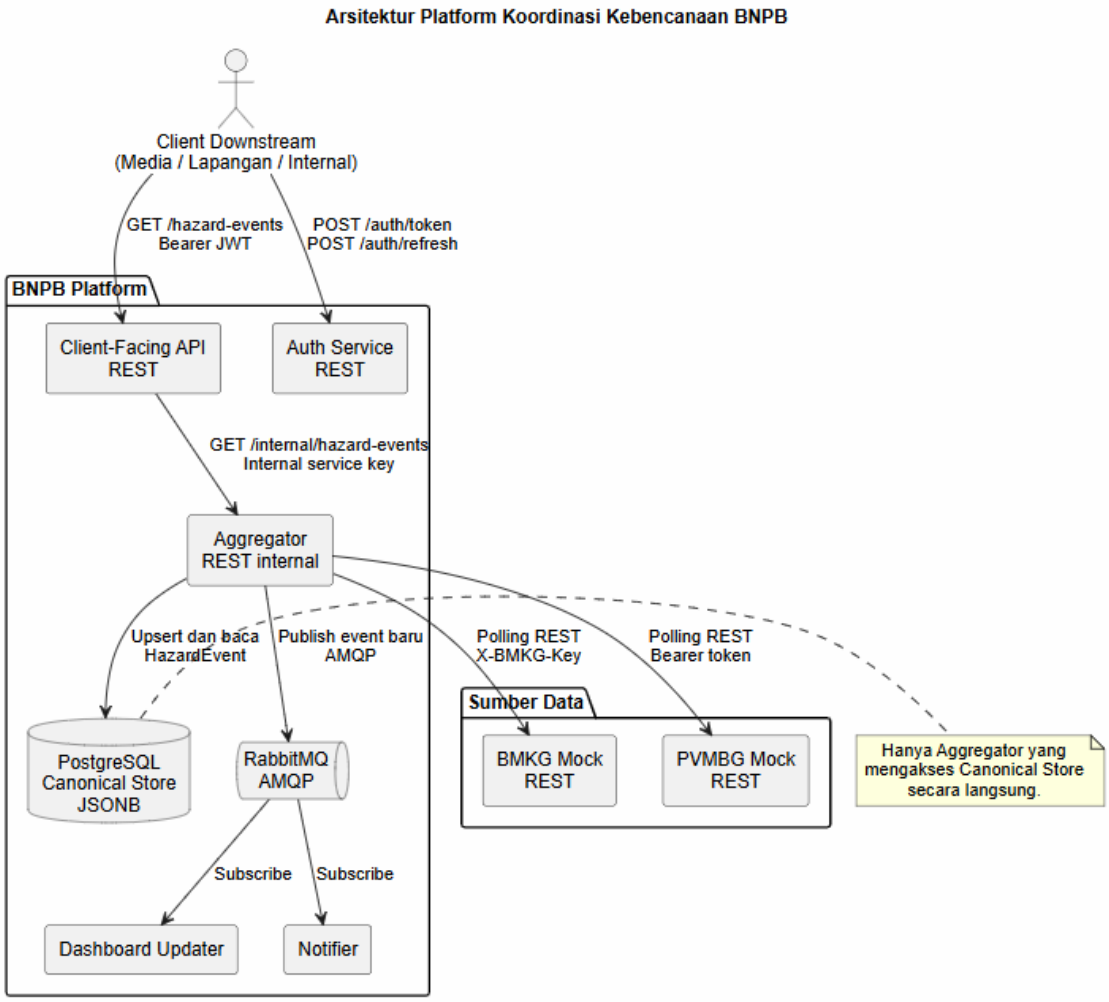
2 Arsitektur Sistem

2.1 Gambaran Umum

Sistem dibangun sebagai sekumpulan service yang dikemas dan dijalankan terpisah melalui Docker Compose. BMKG dan PVMBG berperan sebagai *mock service*. Aggregator melakukan

polling ke kedua sumber, memetakan respons ke format kanonik *HazardEvent*, menyimpannya pada Canonical Store, dan menerbitkan event baru ke message broker.

Client downstream tidak membaca database secara langsung. Permintaan data masuk melalui Client-Facing API, yang memverifikasi token client, menerapkan filter hak akses, lalu mengambil data dari API internal milik Aggregator. Event baru diterima consumer independen untuk kebutuhan dashboard dan notifikasi. Diagram arsitektur sistem ditunjukkan pada Gambar 1.



Gambar 1: Arsitektur platform koordinasi kebencanaan BNPB.

2.2 Komponen dan Batas Service

Tabel 1: Komponen utama sistem.

Komponen	**Antarmuka**	**Tanggung Jawab**
BMKG Mock	REST	Menyediakan data *SeismicEvent* dan *TsunamiWarning*; akses menggunakan `X-BMKG-Key`.
PVMBG Mock	REST	Menyediakan *VolcanicReport*, perubahan skema, dan simulasi outage; akses menggunakan *Bearer token*.
Aggregator	REST internal	Polling, mapping, penyimpanan Canonical Store, serta publikasi event baru.
PostgreSQL	Koneksi database internal	Menyimpan *HazardEvent* dengan atribut dinamis pada kolom `JSONB`; hanya Aggregator yang mengaksesnya langsung.
Client-Facing API	REST	Menyajikan data untuk client, memvalidasi JWT, dan menerapkan RBAC.
Auth Service	REST	Menerbitkan dan memperbarui token client.
RabbitMQ	AMQP	Broker distribusi event dengan pola fan-out.
Event Consumer	AMQP consumer	Menerima event baru secara independen untuk dashboard dan notifikasi.

2.3 Alur Komunikasi dan Data

Aggregator melakukan polling berkala melalui REST. Kredensial untuk BMKG berupa API key, sedangkan PVMBG menggunakan *Bearer token*. Respons sumber dipetakan ke *HazardEvent* dan disimpan oleh Aggregator ke PostgreSQL. Field dinamis disimpan pada `attributes` bertipe `JSONB`.

Setiap event baru yang tersimpan diterbitkan Aggregator ke RabbitMQ. Dashboard Updater dan Notifier berlangganan melalui broker; Aggregator tidak memanggil consumer secara langsung. Untuk alur baca, client memperoleh access token dari Auth Service dan mengirimkannya ke Client-Facing API. Setelah memvalidasi token dan scope, Client-Facing API memanggil endpoint internal Aggregator dengan *internal service key*. Dengan demikian, PostgreSQL bukan dependency langsung bagi Client-Facing API maupun consumer.

2.4 Teknologi dan Konfigurasi

Service aplikasi dibangun menggunakan Node.js, TypeScript, dan Fastify. Komunikasi sinkron menggunakan REST, sedangkan distribusi event memakai RabbitMQ melalui AMQP. Canonical Store menggunakan PostgreSQL dengan `JSONB` untuk menyimpan atribut sumber yang dinamis tanpa penambahan kolom baru.

Seluruh service dijalankan melalui `docker-compose.yml`. Port, URL service, credential, interval polling, parameter circuit breaker, koneksi database, koneksi broker, dan kunci

internal dikonfigurasi melalui environment variable.

3 Problem 1: Arsitektur dan Interoperabilitas Data

Pertanyaan Utama

Bagaimana Aggregator BNPB dapat menyatukan data BMKG dan PVMBG ke format kanonik yang tetap valid ketika PVMBG menambahkan field baru tanpa restart atau redeploy?

3.1 Rancangan Solusi

Sistem menggunakan pola integrasi *pull* melalui REST. Aggregator melakukan polling ke BMKG untuk data *SeismicEvent* dan *TsunamiWarning*, serta ke PVMBG untuk data *VolcanicReport*. Setiap respons dipetakan menjadi kontrak internal tunggal, yaitu *HazardEvent*, sebelum disimpan pada Canonical Store PostgreSQL dan didistribusikan sebagai event. Dengan demikian, service downstream tidak perlu memahami bentuk data khusus milik masing-masing instansi.

3.1.1 Aturan Pemetaan ke HazardEvent

Tabel 2 merangkum aturan pemetaan yang diterapkan oleh Aggregator. Field inti yang diperlukan oleh platform ditempatkan pada kolom kanonik, sedangkan detail sumber ditempatkan pada `attributes`.

Sumber	Data masukan	Pemetaan ke *HazardEvent*
BMKG	*SeismicEvent* dan *TsunamiWarning* terkait	`source=BMKG`, `source_ref_id=event_id`, `hazard_type=SEISMIC`. Nama wilayah dan koordinat diambil dari data gempa. Tingkat keparahan mengikuti peringatan tsunami bila tersedia; jika tidak tersedia, magnitude $\geq$ 6,5 dipetakan menjadi `SIAGA`, magnitude $\geq$ 5,0 menjadi `WASPADA`, dan sisanya `NORMAL`.
PVMBG	*VolcanicReport*	`source=PVMBG`, `source_ref_id=report_id`, `hazard_type=VOLCANIC`. `alert_level` dipetakan ke severity `NORMAL`, `WASPADA`, `SIAGA`, atau `AWAS`. Nama serta koordinat gunung diperoleh dari tabel referensi gunung api milik Aggregator.

Tabel 2: Aturan pemetaan data sumber ke format kanonik.

3.1.2 Kebijakan Evolusi Skema

Mekanisme yang dipilih adalah JSON dengan kebijakan *additive-only*. Field inti yang sudah dikenal dipetakan secara eksplisit untuk menentukan identitas hazard, jenis hazard, tingkat keparahan, lokasi, dan waktu kejadian. Setiap field tambahan dari sumber yang bukan field inti diteruskan tanpa transformasi ke objek `attributes`. Objek ini disimpan pada kolom `JSONB` di Canonical Store. Oleh karena itu, penambahan field seperti `confidence_level` oleh PVMBG tidak memerlukan perubahan kolom tabel, restart, maupun redeploy service.

Kebijakan penanganan field tak dikenal yang dideklarasikan pada sistem ini adalah **meneruskan field ke `attributes`**, bukan mengabaikannya. Konsistensi kebijakan tersebut dibuktikan melalui perubahan skema PVMBG; setelah field `confidence_level` diaktifkan, field tersebut tetap tersedia pada `attributes` dari *HazardEvent* hasil Aggregator.

3.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
Kontrak kanonik *HazardEvent*	Client dan consumer cukup memahami satu bentuk data meskipun sumber data heterogen.	Aggregator menjadi tempat pemeliharaan aturan mapping. Aturan dipisahkan pada mapper per sumber dan diuji secara terpisah.
JSONB dengan kebijakan *additive-only*	Field baru dapat diterima tanpa migrasi database ataupun redeploy; informasi sumber tidak hilang.	Validasi field dinamis lebih longgar dan query atribut tertentu lebih kompleks daripada kolom relasional. Field yang dibutuhkan oleh proses inti tetap dipertahankan sebagai field kanonik yang tervalidasi.
Mapping eksplisit untuk field inti, `attributes` untuk sisanya	Semantik inti seperti severity dan lokasi konsisten, sementara evolusi aditif tetap fleksibel.	Perubahan makna atau tipe field inti tidak dapat ditangani otomatis dan perlu perubahan kontrak serta mapper.

Tabel 3: Keputusan desain dan trade-off untuk interoperabilitas data.

Perubahan skema yang didukung secara sadar adalah penambahan field opsional pada objek laporan PVMBG dan BMKG. Perubahan yang tidak didukung tanpa perubahan kode adalah penghapusan, penggantian nama, perubahan tipe, atau perubahan makna dari field inti; misalnya `report_id`, `volcano_id`, `alert_level`, dan `reported_at` pada PVMBG. Sistem juga mengharapkan envelope respons tetap berbentuk `{data: [...]}`. Perubahan yang melanggar kontrak tersebut diperlakukan sebagai perubahan *breaking* dan harus disertai pembaruan mapper serta pengujian kompatibilitas.

3.3 Proof of Concept (PoC)

Langkah berikut digunakan untuk menunjukkan bahwa pemetaan kedua sumber berjalan dan perubahan skema aditif dapat diterima saat sistem aktif.

1. Nyalakan seluruh sistem dan pastikan setiap container sehat.

```
docker compose up -d --build
docker compose ps
```

2. Pastikan PVMBG menggunakan skema awal dengan menonaktifkan field tambahan. Endpoint admin ini hanya dipakai untuk demo/pengujian.

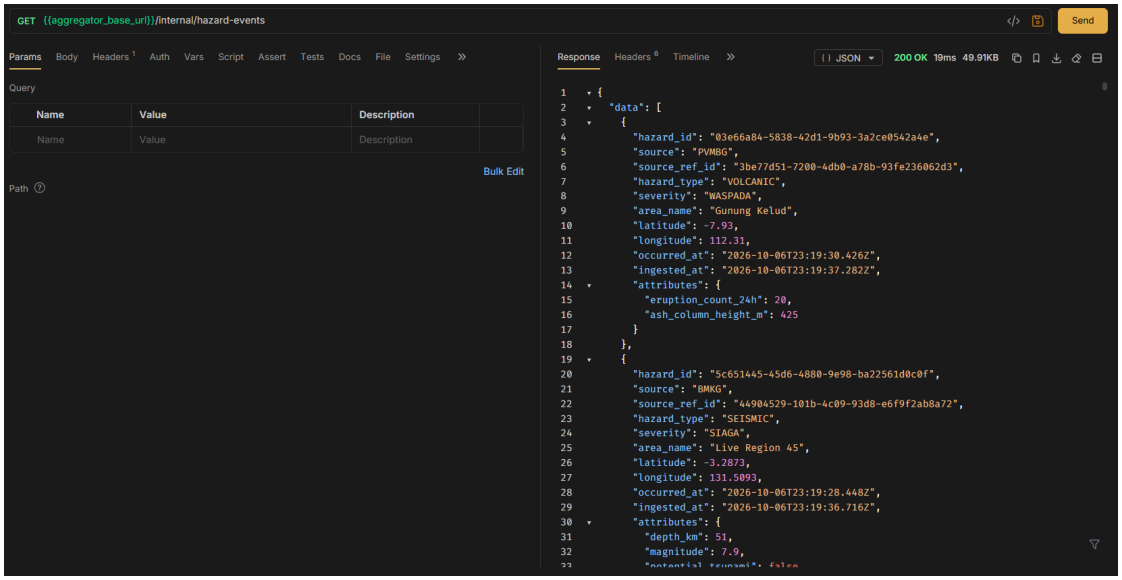
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": false}'
```



Gambar 2: Status seluruh container setelah menonaktifkan field tambahan PVMBG.

3. Tunggu satu atau dua interval polling, kemudian ambil data kanonik dari API internal Aggregator. Respons harus memperlihatkan hazard dari kedua sumber dengan `source` masing-masing `BMKG` dan `PVMBG`.

```
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events" `
  -Headers $headers
```



Gambar 3: Respons API internal Aggregator menampilkan data dari BMKG dan PVMBG.

4. Aktifkan perubahan skema saat semua service tetap berjalan. Tidak ada perintah restart atau redeploy setelah langkah ini.

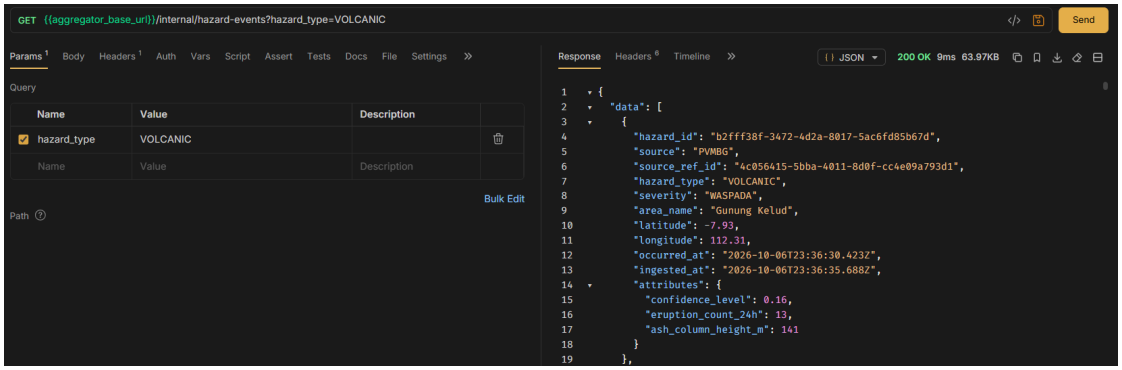
```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:8082/admin/schema-version `
  -ContentType "application/json" `
  -Body '{"enabled": true}'
```



Gambar 4: Mengaktifkan field tambahan PVMBG saat sistem berjalan.

5. Tunggu sedikitnya 15 detik agar PVMBG menghasilkan laporan baru dan Aggregator menyelesaikan polling. Ambil hanya data vulkanik, lalu verifikasi bahwa salah satu objek memiliki `attributes.confidence_level` tanpa ada perubahan schema tabel.

```
Invoke-RestMethod `
  -Uri "http://localhost:8090/internal/hazard-events?hazard_type=VOLCANIC" `
  -Headers $headers
```



Gambar 5: Respons API internal Aggregator setelah perubahan

6. Verifikasi bahwa Aggregator tetap sehat dan tidak mengalami crash.

```
docker compose ps aggregator
```

```
PS C:\File\ITB\Semester-7\aat\AAT-bebasss> docker compose ps aggregator
NAME                                IMAGE                                                                                  COMMAND
      SERVICE      CREATED          STATUS                    PORTS
aat-bebasss-aggregator-1   sha256:70adac4c61b16d1f854f85bf6a59cf51855dc6e43e2316be71d95062f3819877   "docker-entrypoint.
s…"   aggregator   10 minutes ago   Up 9 minutes (healthy)   0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp
PS C:\File\ITB\Semester-7\aat\AAT-bebasss>
```

Gambar 6: Kondisi tetap sehat

Dengan proof tersebut, kriteria penyelesaian terpenuhi, yaitu sistem memetakan kedua sumber pada skema awal, menerima perubahan skema aditif ketika sedang berjalan, dan mempertahankan field tambahan secara konsisten di `attributes`.

4 Problem 2: Concurrency, Availability, dan Graceful Degradation

Pertanyaan Utama

Bagaimana sistem tetap responsif saat BMKG dan PVMBG memiliki latensi berbeda, menangani lonjakan permintaan, dan tetap melayani data yang tersedia ketika salah satu sumber mengalami outage?

4.1 Rancangan Solusi

Problem ini memisahkan jalur pembaruan data dari jalur pembacaan client. Aggregator mengambil data BMKG dan PVMBG secara berkala, menyimpannya pada Canonical Store, sedangkan Client API membaca data yang sudah tersedia melalui API internal Aggregator. Dengan pemisahan tersebut, client tidak perlu menunggu panggilan langsung ke sumber eksternal yang lambat pada setiap permintaan `GET /hazard-events`.

4.1.1 Tiga Risiko yang Ditangani

Rancangan ini menangani tiga kegagalan. *Head-of-line blocking* terjadi ketika pengambilan data BMKG dan PVMBG dilakukan sekuensial sehingga sumber cepat tertahan oleh sumber lambat. *Resource exhaustion* terjadi ketika polling tumpang-tindih atau panggilan ke sumber lambat tidak dibatasi. Ketiadaan degradasi terkendali terjadi ketika satu sumber gagal dan seluruh respons client ikut gagal, padahal data terakhir masih tersedia dan berguna.

4.1.2 Model Konkurensi dan Isolasi Sumber

Model konkurensi yang dipilih adalah asynchronous I/O Node.js. Dalam satu siklus, BMKG dan PVMBG diproses sebagai operasi asynchronous yang berjalan paralel. Satu siklus polling tidak ditumpuk dengan siklus berikutnya.

Dalam satu siklus polling, Aggregator memulai polling BMKG dan PVMBG secara paralel. Hasil keduanya ditunggu secara independen menggunakan *all-settled semantics*: kegagalan satu sumber tidak membatalkan hasil sumber lain. Siklus berikutnya tidak ditumpuk ketika siklus sebelumnya masih berjalan. Pendekatan ini mencegah ledakan request ke sumber eksternal ketika PVMBG memiliki latensi tinggi.

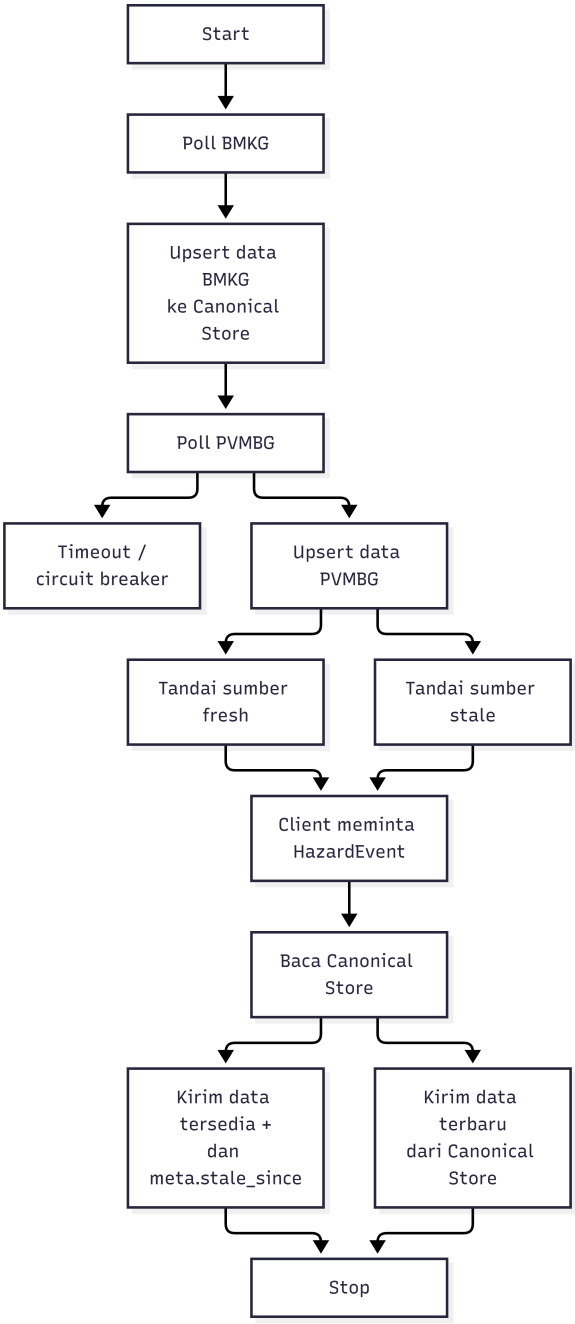
Setiap panggilan keluar mempunyai timeout yang dikonfigurasi melalui `SOURCE_REQUEST_TIMEOUT_MS` serta mencatat `latency_ms` pada log terstruktur. Untuk PVMBG, client sumber juga dibungkus oleh *circuit breaker* dan *concurrency limiter*. Circuit breaker membuka rangkaian setelah sejumlah kegagalan berturut-turut, menolak request selama masa *cooldown*, lalu mengizinkan satu percobaan pemulihan. Concurrency limiter membatasi jumlah panggilan PVMBG yang aktif sesuai `PVMBG_MAX_CONCURRENT_CALLS`.

4.1.3 Graceful Degradation dan Data Stale

Jika polling suatu sumber gagal, Aggregator tidak menghapus event yang telah tersimpan. Sebaliknya, sumber tersebut diberi penanda stale pada tabel `source_sync_state`; respons pembacaan memuat `meta.stale_since`. Dengan demikian, client masih memperoleh data terakhir yang diketahui sistem, sekaligus mengetahui bahwa sinkronisasi salah satu sumber sedang bermasalah. Setelah polling sumber kembali berhasil, penanda stale dihapus tanpa restart service.

Untuk mengurangi beban polling ketika histori bertambah, Aggregator menyimpan waktu sinkronisasi sukses terakhir per sumber. Polling pertama mengambil data yang tersedia, sedangkan polling berikutnya memakai parameter `since` dengan overlap yang dapat dikonfigurasi melalui `POLL_LOOKBACK_MS`. Event dalam overlap aman diproses ulang karena penyimpanan menggunakan upsert berdasarkan pasangan `source` dan `source_ref_id`.

Keputusan sistem ketika polling PVMBG berhasil atau gagal dirangkum pada Gambar 7. Diagram tersebut menunjukkan bahwa kegagalan PVMBG tidak membatalkan pembaruan BMKG maupun pembacaan data yang telah tersedia bagi client.



Gambar 7: Keputusan graceful degradation ketika salah satu sumber data gagal atau melampaui batas waktu.

4.2 Alternatif dan Trade-off

Keputusan	Keuntungan	Trade-off dan mitigasi
Polling paralel BMKG dan PVMBG	Latensi PVMBG tidak menunda dimulainya pengambilan data BMKG dalam siklus yang sama.	Siklus dianggap selesai setelah kedua polling selesai. Sistem mencegah siklus tumpang-tindih agar sumber tidak dibanjiri request.
Canonical Store sebagai read model	Pembacaan client tidak bergantung pada latensi sumber eksternal dan tetap dapat dilayani saat salah satu sumber outage.	Data bersifat *eventually consistent*; data yang dibaca dapat merupakan data terakhir, bukan data real-time dari sumber. Penanda `stale_since` menyatakan kondisi tersebut secara eksplisit.
Timeout, circuit breaker, dan concurrency limiter pada PVMBG	Kegagalan atau kelambatan berulang dari PVMBG tidak terus-menerus mengonsumsi waktu dan koneksi Aggregator.	Data PVMBG dapat tertunda selama circuit breaker terbuka. Data terakhir tetap disajikan dan polling pemulihan dilakukan setelah cooldown.
Incremental polling dengan overlap	Volume data yang diambil ulang berkurang dibandingkan mengambil seluruh histori pada setiap interval.	Event yang terlambat lebih lama daripada overlap dapat terlewat. Overlap dikonfigurasi dan upsert mencegah data overlap menjadi duplikat.

Tabel 4: Keputusan desain dan trade-off untuk concurrency serta availability.

4.3 Proof of Concept (PoC)

4.3.1 Uji Latensi BMKG-only

Pengujian ini membuktikan bahwa keterlambatan PVMBG tidak menyebabkan *head-of-line blocking* pada request yang hanya membutuhkan data BMKG. Langkah pembuktiannya adalah sebagai berikut.

1. Atur `PVMBG_DELAY_MIN_MS=3000` dan `PVMBG_DELAY_MAX_MS=3000` pada `.env`, kemudian recreate hanya service PVMBG.

```
docker compose up -d --force-recreate pvmbg-mock
```

2. Jalankan skenario *k6*. Skenario ini mengirim request BMKG-only dan request vulkanik secara bersamaan.

```
k6 run tests/phase6/p2-concurrency.k6.js
```

3. Catat p95 workload BMKG-only. Hasilnya dirangkum pada Tabel 5.

Aspek	Hasil	Kriteria
Kondisi PVMBG	Delay tetap 3.000 ms	Delay PVMBG 3 detik
Workload bersamaan	50 VU BMKG-only dan 10 VU vulkanik	Kedua workload berjalan bersamaan
p95 latensi BMKG-only	231,59 ms	< 300 ms
Error rate BMKG-only	0,00%	$< 1\%$
Status threshold BMKG-only	Lulus	Seluruh threshold lulus

Tabel 5: Hasil uji latensi request BMKG-only ketika PVMBG lambat.

Hasil tersebut menunjukkan bahwa keterlambatan PVMBG tidak menyebabkan *head-of-line blocking* pada pembacaan data seismik.

4.3.2 Uji Beban Berkelanjutan

Tooling yang digunakan adalah *k6* dengan script `tests/phase6/p2-concurrency.k6.js`. Script menjalankan 50 virtual user untuk request BMKG-only dan 10 virtual user untuk request vulkanik selama 60 detik secara sustained. Output pengujian harus melaporkan throughput, p50, p95, p99, dan error rate. Sistem dinyatakan memenuhi kriteria apabila error rate, di luar penolakan terkontrol seperti HTTP 429, berada di bawah 1% dan tidak terjadi crash.

```
k6 run tests/phase6/p2-concurrency.k6.js
```

Hasil pengujian ditunjukkan pada Tabel 6. Nilai p50 ditampilkan oleh *k6* sebagai `med`. Metrik berlabel HTTP dihitung dari seluruh request pada pengujian.

Metrik	Hasil	Kriteria
Konfigurasi beban	50 VU BMKG-only dan 10 VU vulkanik selama 60 detik	$\geq$50 koneksi paralel selama $\geq$60 detik
Total request	20.244 request	–
Throughput	336,74 request/detik	–
p50 latensi HTTP keseluruhan	165,95 ms	Dilaporkan
p95 latensi HTTP keseluruhan	231,27 ms	Dilaporkan
p99 latensi HTTP keseluruhan	333,48 ms	Dilaporkan
Error rate	0,00% (0 dari 20.244 request)	$< 1\%$
Check skenario	37.049 berhasil, 0 gagal	Seluruh check berhasil

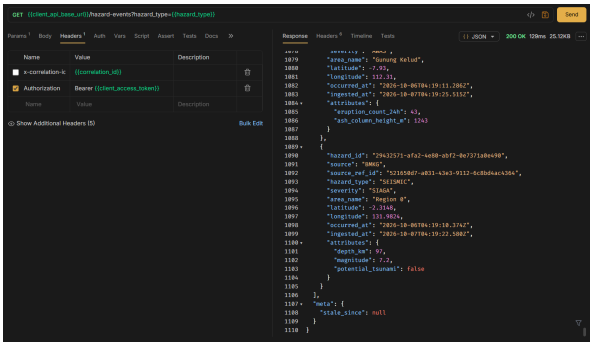
Tabel 6: Hasil load testing sustained menggunakan *k6*.

Dengan error rate 0,00%, pengujian memenuhi batas keandalan yang ditetapkan. Selama pengujian, seluruh service tetap berjalan dan tidak ada penolakan terkontrol HTTP 429 yang tercatat.

4.3.3 Outage dan Pemulihan PVMBG

Pengujian ini membuktikan bahwa kegagalan PVMBG tidak menghentikan jalur pembacaan client dan bahwa sistem pulih tanpa restart service.

1. Kirim request `CLIENT API - /hazard-events` dengan `hazard_type=VOLCANIC`. Terlihat bahwa request berhasil, berisi data vulkanik, dan `meta.stale_since` bernilai `null`.



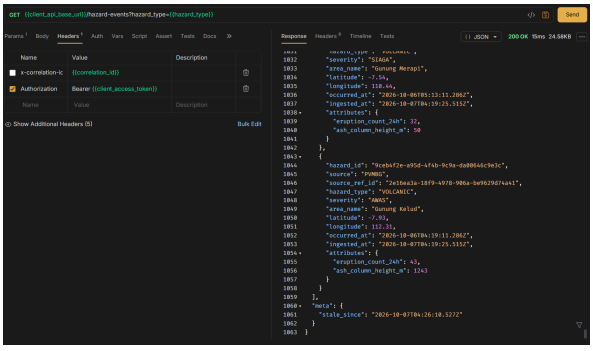
Gambar 8: Kondisi awal saat service aktif dan `stale_since` bernilai `null`.

2. Set variable `outage_enabled=true` dan `outage_duration_seconds=30`, lalu jalankan request `PVMBG MOCK - /admin/outage`. Respons menyatakan `outage_active=true`.



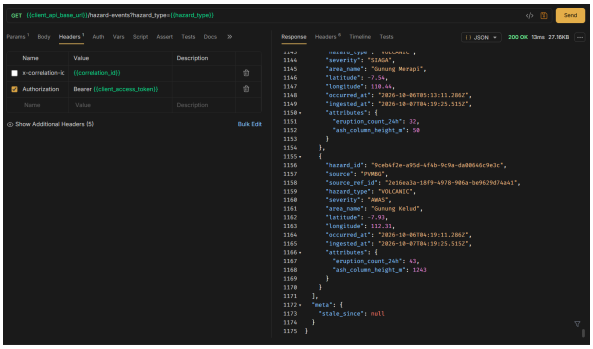
Gambar 9: Membuat server down sesaat

3. Tunggu sekitar 8 detik agar polling yang sedang berlangsung selesai dan Aggregator menjalankan polling PVMBG yang gagal. Kemudian jalankan kembali request `CLIENT API - /hazard-events` untuk `hazard_type=VOLCANIC`. Respons vulkanik memuat data terakhir dari Canonical Store, sedangkan `meta.stale_since` berisi waktu sejak kapan agregrator mendeteksi kegagalan sinkronisasi.



Gambar 10: `stale_since` terisi saat PVMBG mengalami outage.

4. Setelah durasi outage selesai, tunggu sekitar 8 detik agar satu polling pemulihan berhasil, lalu ulangi request vulkanik. Respons `meta.stale_since` kembali `null`.



Gambar 11: `stale_since` kembali `null` setelah PVMBG pulih.

5 Problem 3: Service-to-Service Authentication dan Trust

Pertanyaan Utama

Bagaimana setiap domain kepercayaan dipisahkan sehingga kredensial antarsumber tidak dapat dipertukarkan, sementara hak akses client downstream tetap ditegakkan berdasarkan scope dan masa berlaku token?

5.1 Rancangan Solusi

5.2 Alternatif dan Trade-off

5.3 Batasan dan Bukti Penyelesaian

6 Problem 4: Arsitektur Microservice dan Flexible Storage

Pertanyaan Utama

Bagaimana sistem dipecah menjadi layanan yang dapat dijalankan independen dan menyimpan atribut data yang berubah tanpa migrasi schema atau akses storage lintas service?

6.1 Rancangan Solusi

6.2 Alternatif dan Trade-off

6.3 Batasan dan Bukti Penyelesaian

7 Problem 5: Fan-out Event Distribution dan Pub-sub

Pertanyaan Utama

Bagaimana HazardEvent baru didistribusikan ke banyak consumer tanpa producer memanggil consumer secara langsung atau terhambat ketika salah satu consumer mati?

7.1 Rancangan Solusi

7.2 Alternatif dan Trade-off

7.3 Batasan dan Bukti Penyelesaian

A Bukti Pengujian Tambahan